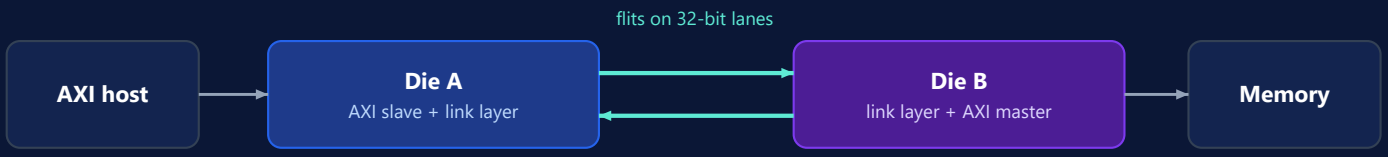
PORTFOLIO PROJECT | RTL DESIGN + VERIFICATION

D2D-Link

A UCIe-inspired chiplet die-to-die link controller, explained step by step.

AXI4 traffic from Die A to Die B over a flit-based link with CRC-16, Go-Back-N retry, credit flow control on two virtual channels, link training, and three asynchronous clock domains. Synthesizable SystemVerilog, verified with open-source tools only.



44 / 44	5,108	3,604	100%	4	0
regression jobs pass	AXI transactions checked	faulted flits recovered	functional coverage (233 bins)	unbounded formal proofs	lint warnings

What this document walks through

1. **The big picture** — What a die-to-die link is, what D2D-Link does
2. **The flit** — How data is packed, framed and CRC-protected
3. **Reliability** — Go-Back-N retry, replay buffer, credits, virtual channels
4. **Training and clocks** — Link state machine and safe clock crossings
5. **A transaction's journey** — One AXI write followed hop by hop
6. **Verification** — pyuvm testbench, fault injection, coverage, formal
7. **Results and lessons** — The bug found, synthesis estimates, limitations
8. **Design Q&A** — Common design questions, answered from the RTL

Tools: Verilator 5.020 | cocotb 1.9.2 | pyuvm 3.0.0 | cocotb-coverage 1.2.0 | SymbiYosys + z3 / ABC PDR | Yosys 0.33 | Verible | SKY130 HD

Author: Pranay

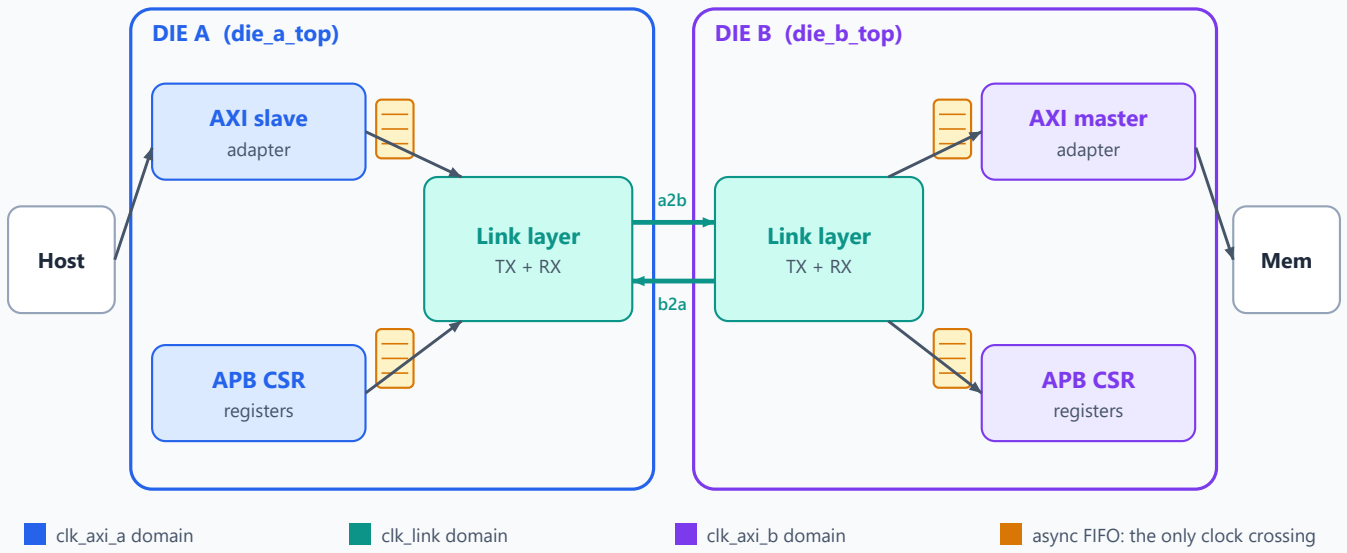
Measured numbers come from log files in the repository (docs/results.md).

The big picture

Why die-to-die links exist, and what this controller does

Modern processors are increasingly built from **chiplets**: several smaller dies in one package instead of one huge die. The dies still have to talk to each other as if they were one chip. A **die-to-die (D2D) link controller** does that job. It takes on-chip bus traffic on one die, packs it into small fixed-size packets called **flits**, sends them across a few wires, and rebuilds the bus traffic on the other die. Industry standards such as UCIe define this. D2D-Link is a smaller design built on the same ideas, for a host on Die A that reads and writes a memory on Die B.

System architecture



Three layers, bottom to top

Protocol layer
AXI adapters turn bus channels (AW, W, B, AR, R) into flit bodies and back again.

Link layer
Adds sequence numbers, CRC, retry, credits and training. Delivers flits in order, with no loss.

Physical lane model
A 32-bit lane, 4 beats per flit. The testbench injects bit flips and drops here.

What the design supports

AXI4 subset	64-bit data, 32-bit addr, 4-bit ID
Bursts	INCR, 1 to 16 beats, full WSTRB
Outstanding	4 writes + 4 reads at Die A
Ordering	strictly in order (documented)
Flit	128 bits, CRC-16/CCITT
Retry	Go-Back-N, 16-entry replay buffer
Flow control	credits on 2 virtual channels
Clocks	3 asynchronous domains
Control	APB3 CSRs + interrupts per die

Glossary: eight words you will meet on every page

Flit
fixed 128-bit packet, the unit the link moves

CRC
16-bit checksum that detects corrupted flits

SEQ / ACK
flit numbering, and "I got everything before N"

NAK
"something was bad, resend from N"

Credit
permission to use one free slot in the receiver

Virtual channel
independent lane of credits (REQ, RSP)

CDC
moving data safely between unrelated clocks

Gray code
counter encoding where one bit changes per step

STEP 2

The flit

The 128-bit packet that carries everything across the link

Every piece of information crossing the link travels in a **flit** (flow-control unit), a fixed 128-bit packet. A fixed size keeps the hardware simple: the receiver always knows where a flit ends, and the CRC always covers the same bits. Each flit has a header, a 72-bit payload and a 16-bit CRC.

Bit layout

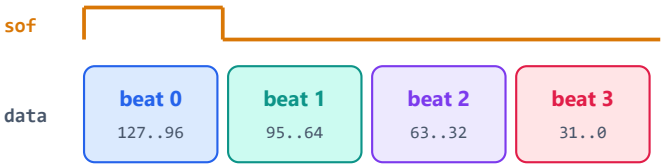


- **TYPE** — which kind of flit (table below)
- **ACK_SEQ** — next SEQ expected from the other side
- **FLAGS** — LAST beat of a burst, or LOCKED in training
- **SEQ** — sequence number, used by retry
- **CRED_*** — running count of freed receive slots, per channel
- **CRC** — error check over bits 127..16

Nine flit types

Code	Type	Retried	Carries
1	REQ_HDR	yes	AW or AR: address, ID, length
2	REQ_DATA	yes	W beat: 64-bit data + strobes
3	RSP_HDR	yes	B response: ID, resp
4	RSP_DATA	yes	R beat: 64-bit data, ID, resp
0, 5, 7	IDLE, ACK, CREDIT	no	filler; still carries ACK + credits
6	NAK	no	resend from ACK_SEQ
8	TRAIN	no	fixed pattern + LOCKED bit

Four beats on a 32-bit lane



One beat per **clk_link** cycle, MSB first. `sof` marks beat 0 so the receiver can realign; a truncated flit counts as an error and triggers a NAK. The link always sends something: when there is no data it sends IDLE, ACK or CREDIT flits, which keeps ACKs and credits flowing.

What the 72-bit payload holds

Flit type	Payload fields (bit 87 down to 16)	Used for
REQ_HDR	`is_write | ID[4] | address[32] | len[4] | size[3] | burst[2]`	one AXI AW or AR
REQ_DATA	`wdata[64] | wstrb[8] (FLAGS.LAST = last beat)`	one AXI W beat
RSP_HDR	`bid[4] | bresp[2]`	one AXI B response
RSP_DATA	`rdata[64] | rid[4] | rresp[2] (FLAGS.LAST = RLAST)`	one AXI R beat
TRAIN	`fixed 72-bit pattern 0xD2D2_A5A5_5A5A_C3C3_3C`	link training

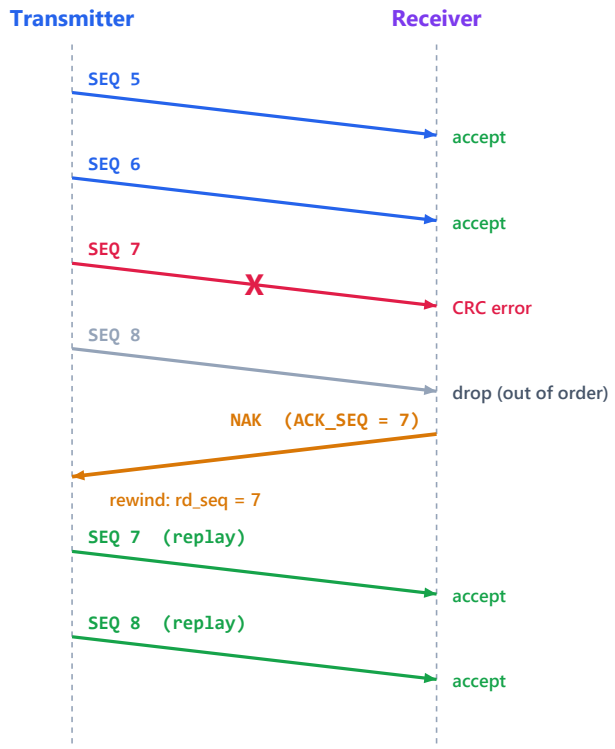
CRC-16: catching corrupted flits

The transmitter runs the 112 header + payload bits through a **CRC-16/CCITT** (polynomial 0x1021, start value 0xFFFF) and appends the 16-bit result. The receiver recomputes it; a mismatch means the flit was damaged, so it is dropped and a NAK is sent. CRC-16 is guaranteed to catch any 1, 2 or 3 flipped bits and any burst of up to 16 bits. In hardware it is one combinational XOR tree (`rtl/common/crc16.sv`), checked against a Python model on 1,004 random and corner-case vectors.

Reliability: retry and credits

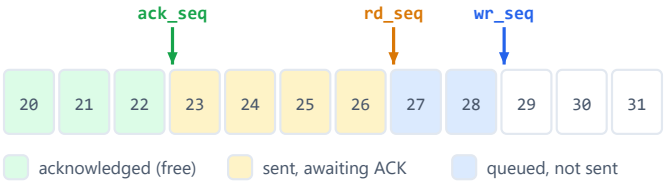
How the link delivers every flit in order, without overflowing the receiver

Go-Back-N retry, in action



Every data flit gets a **sequence number**, and a copy stays in the **replay buffer** until it is acknowledged. When the receiver sees a bad or out-of-order flit it sends one **NAK**, and the transmitter goes back and resends everything from that point. If a flit vanishes and nothing follows it, a **replay timeout** triggers the same rewind.

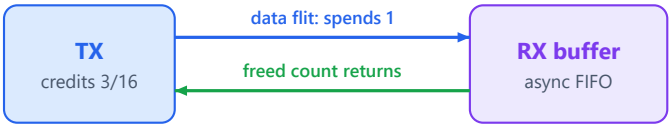
The replay buffer: three pointers



The rule is `ack ≤ rd ≤ wr` and `wr - ack ≤ 16`. There are 256 sequence numbers and the window is 16, less than half, so the receiver can always tell a **duplicate** (drop it) from a **gap** (NAK it). This invariant is **formally proven**.

Credits: never overflow the receiver

A **credit** is one free slot in the receiver's buffer. The transmitter may only send while it holds a credit; the receiver hands credits back as its buffer drains.



Three design decisions worth remembering

Cumulative, not incremental

ACK and credit fields carry **running totals** (mod 256). If the flit carrying them is lost, the next good flit has the up-to-date total, so a credit or ACK can never leak away.

Credits are spent once

A credit is consumed when a flit enters the replay buffer, not each time it is sent. Replays never spend credits again, so an in-order flit always finds a free slot (asserted in RTL).

Two virtual channels

Requests and responses have **separate credits**. A jammed request path can never block the responses that would clear it, which removes protocol deadlock.

What triggers a replay, and how often it happened in the final regression

Trigger	Detected by	Recovery	Count (44 runs)
Corrupted flit	CRC mismatch at the receiver	NAK, then Go-Back-N replay	350 NAKs sent
Lost flit followed by more data	SEQ gap at the next flit	NAK, then replay	(included above)
Lost last flit, lost NAK or ACKs	replay timer expires	replay from oldest unACKed	62 timeouts
Error burst above threshold	error-window counter	retrain, then replay	26 retrains
All of the above	-	flits actually resent	353 replays

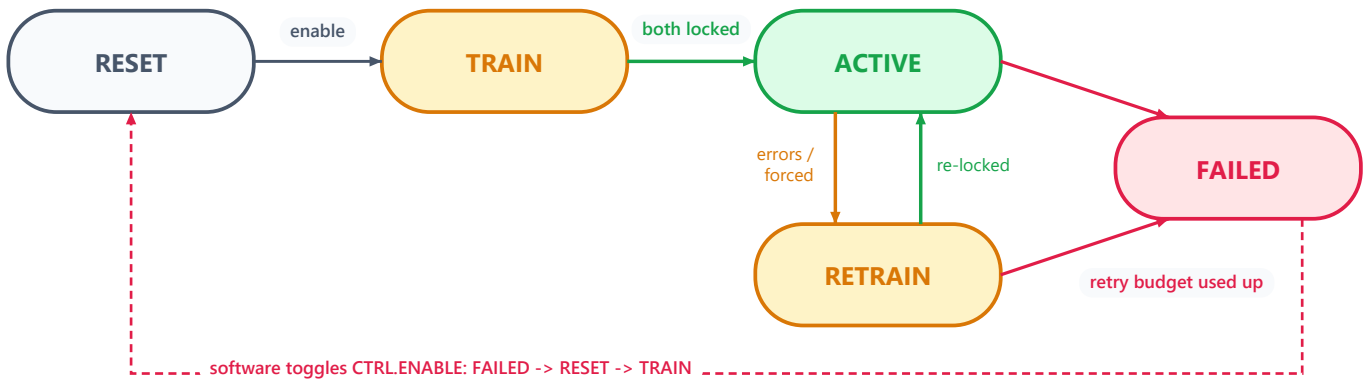
STEP 4

Link training and clock domains

Bringing the link up safely, and moving data between unrelated clocks

The link training state machine

Before any data flows, both dies must agree the link works. Each side sends **TRAIN** flits; after 8 good ones in a row it is **locked** and says so in its own TRAIN flits. When both sides are locked, the link goes **ACTIVE**. Too many errors send it back to **RETRAIN**; too many failed attempts end in **FAILED**.



No data is lost across a retrain

Retraining never touches the replay buffer, sequence numbers, credits or receive FIFOs. On return to ACTIVE the transmitter replays everything still unacknowledged and the receiver drops duplicates by sequence number. A TRAIN flit with LOCKED=1 arriving while ACTIVE is ignored: without that rule the two dies could keep knocking each other back into retraining forever.

Knob (default)	Meaning	Knob (default)	Meaning
ERR_THRESH = 8	retrain if > 8 bad flits in a window	TRAIN_GOOD = 8	good TRAIN flits needed to lock
ERR_WINDOW = 256	window length, in received flits	TRAIN_TIMEOUT = 1024	cycles per training attempt
MAX_RETRAIN = 4	attempts before FAILED	STABLE_CYCLES = 2048	ACTIVE time that resets the attempt count

Clock-domain crossing (CDC)

The two AXI sides and the link run on **three unrelated clocks**. Sampling a changing multi-bit value from another clock can capture a half-updated number. The rule here is strict: **every crossing is an async FIFO with gray-coded pointers**, and nothing else crosses.

Why gray code?

Step	Binary	Gray	bits that change
3 to 4	`011 to 100`	`010 to 110`	binary 3, gray 1
7 to 0	`111 to 000`	`100 to 000`	binary 3, gray 1

If the reading clock samples mid-change, a gray pointer is always either the old or the new value, never a random third one. The FIFO was **formally proven** with two unconstrained clocks.

The 8 crossings

#	Die	Direction	Carries
1	A	axi_a to link	requests (AW / W / AR flits)
2	A	link to axi_a	responses; RSP credit buffer
3, 7	A, B	axi to link	CSR config messages
4, 8	A, B	link to axi	status + counter snapshots
5	B	link to axi_b	requests; REQ credit buffer
6	B	axi_b to link	responses (B / R flits)

Counters cross as one coherent snapshot through a FIFO, never bit by bit. **Credit return** reuses the receive FIFO's already-synchronized read pointer, so it needs no extra crossing; it lags by 2 to 3 cycles, so credits come back late, never early.

STEP 5

A transaction's journey

Following one AXI write burst from the host to memory and back

1. **Host** — The host drives **AW** (address, ID, length) on Die A. The slave adapter accepts it only if fewer than 4 writes are outstanding, then accepts each **W** beat.
2. **Pack** — The adapter turns AW into one **REQ_HDR** flit body and each W beat into a **REQ_DATA** body (64-bit data + 8 strobe bits), and pushes them into the request async FIFO.
3. **Cross** — The bodies cross from `clk_axi_a` into `clk_link` through the gray-pointer FIFO.
4. **Enqueue** — If a REQ credit is available and the replay buffer has room, the body gets the next **SEQ**, is stored in the replay buffer, and one credit is spent.
5. **Send** — flit_tx adds SEQ, the current ACK and credit counts, computes the **CRC**, and sends the 128 bits as 4 beats on the a2b lane.
6. **Check** — flit_rx on Die B rebuilds the flit and checks CRC and SEQ. In order: accept it into the REQ receive FIFO. Bad or gap: NAK. Duplicate: drop.
7. **Execute** — Die B's master adapter pops the header, drives **AW**, then the **W** beats to memory, and waits for the **B** response. One transaction at a time, in order.
8. **Respond** — The B response becomes an **RSP_HDR** flit and travels back on the b2a lane on the RSP virtual channel, with its own credits.
9. **ACK** — Every flit Die B sends carries ACK_SEQ. When Die A sees it, acknowledged entries leave the replay buffer. As FIFOs drain, the freed-slot counts return credits.
10. **Done** — Die A pops the RSP_HDR and drives **B** to the host; the write retires. Reads work the same way, with R beats returning as RSP_DATA flits.

If step 5 goes wrong

A **corrupted** flit fails the CRC at step 6 and Die B sends a NAK; Die A rewinds and resends from that flit. A **dropped** flit shows up as a SEQ gap at the next flit (NAK), or, if nothing follows it, as a **replay timeout** on Die A. Either way the host just sees a slightly later B response.

Why one transaction at a time at Die B?

Strict ordering makes every response predictable. The scoreboard can compute the exact expected read data and catch any reordering. The cost is throughput, which is documented as a limitation; per-ID reordering would be the next feature.

Control and status registers (APB3, one block per die)

Offset	Register	What it does	Offset	Register	What it does
0x00	CTRL	ENABLE, FORCE_RETRAIN (pulse)	0x1C	ERR_CFG	retrain threshold, retry budget
0x04	STATUS	link state, LINK_UP	0x20	ERR_WINDOW	error window (flits)
0x08-18	5 counters	CRC errors, NAKs, replays, timeouts, retrains	0x24 / 28	IRQ_EN / STATUS	link-down, retrain, failed, CRC (W1C)
0x2C	ID	constant 0xD2D10001	other	-	PSLVERR error response

STEP 6

Verification

Spec first, block tests, a UVM-style system testbench, fault injection and formal proofs

Spec 4 docs → **Unit tests** 9 cases → **System tests** 14 tests → **Regression** 44 jobs → **Formal** 10 tasks → **Lint** 0 warnings → **Synthesis** 12 blocks

The pyuvm testbench

D2dEnv

- **AXI master agent** — Die A host
- **AXI memory agent** — Die B, reactive
- **Lane fault agents x2** — flips, bursts, drops
- **APB agents x2** — CSR access
- **Scoreboard** — reference memory
- **Coverage** — cocotb-coverage

The scoreboard keeps an exact **reference memory**. Because execution is strictly ordered, it predicts every response and read value, and checks that each request reaches Die B exactly once, in order. That catches **loss, duplication, reordering and corruption** under any fault.

14 seeded system tests

smoke · random_rw · max_burst · outstanding_stress · credit_starvation · backpressure_random · crc_error_retry · flit_drop_timeout · error_storm_retrain · link_fail_and_recover · csr_access · ltssm_arcs · clock_ratio_sweep · reset_during_traffic

Red = fault or reset stress. Every test takes a seed; the regression runs all tests over several seeds on all CPU cores under a 15-minute cap and writes JUnit XML.

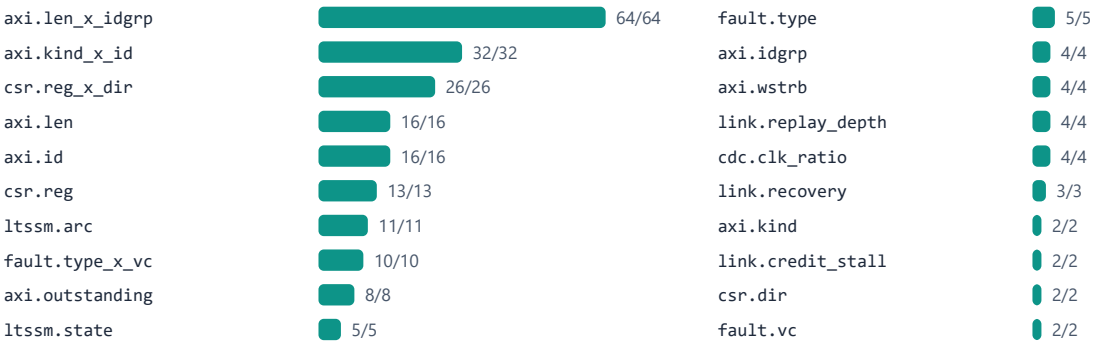
Final regression (from logs)

44/44	5,108	3,604
jobs pass	transactions	faults injected
350	**353**	**62**
NAKs	replays	timeouts

Formal proofs (SymbiYosys, small configuration)

Job	Method	What is proven	Result
Async FIFO	ABC PDR, unbounded	no overflow/underflow, 1-bit gray steps, data order, with two free clocks	PASS
Credits	ABC PDR, unbounded	credits stay in [0, MAX]; conservation even with late or lost credit returns	PASS
Replay buffer	k-induction	ack <= rd <= wr, window <= depth, full never accepts	PASS
Go-Back-N link	ABC PDR, unbounded	delivered stream is exactly 0, 1, 2, ... over a lossy channel	PASS
Training FSM	BMC, depth 20	legal states; ACTIVE only with lock; FAILED only after the budget	PASS

Functional coverage: 233 / 233 bins = 100% (bins per coverage item, all hit)



STEPS 7 + 8

Results, lessons and design Q&A

What was found, what it costs in silicon, and why it is built this way

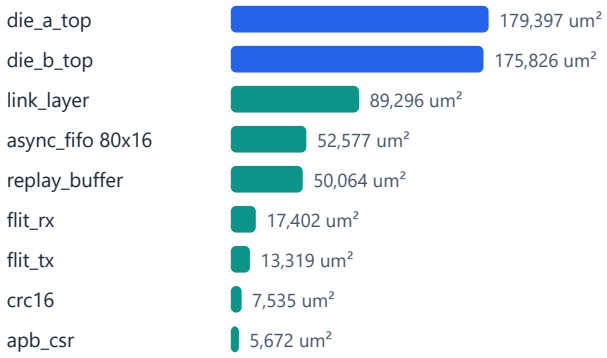
The real RTL bug the tests caught

AXI READY high while the block was still in reset

When reset is released at the pin, each domain's internal reset stays active for 2 more clocks (the synchronizer). In that window the slave's `AWREADY` was built from reset values and read 1. A host starting right after reset had its write "accepted", then silently lost, and the bus hung. **Caught by** `reset_during_traffic`. **Fix:** qualify AWREADY, ARREADY, WREADY and PREADY with the synchronized reset.

Also logged: one design characterization (at a 3% flit error rate the link correctly escalates to FAILED, because the retrain threshold is about 3.1%) and 5 testbench bugs. No check was weakened to make a test pass.

Synthesis estimates (Yosys 0.33 + SKY130 HD)



Standard-cell area only, no routing. The worst combinational path is about **20.8 ns** (replay-buffer mux into the 112-bit CRC), which suggests roughly **48 MHz** in SKY130 at the typical corner. Both are **estimates**: no wires, no clock tree, no OpenSTA. Next steps: pipeline the CRC check and use SRAM instead of about 2,600 flops per die for the buffers.

Common design questions

Q Why cumulative ACK and credit counts?

An increment lost with a corrupted flit is gone for good: a credit leaks and the link eventually deadlocks. A running total heals itself: the next good flit carries it and the transmitter takes the difference.

Q Why keep the retry window under half the SEQ space?

The receiver classifies (seq - expected) mod 256: below 128 is a gap (NAK), otherwise a duplicate (drop). A window of 16 keeps the two cases from overlapping.

Q How do credits cross clock domains safely?

The receive buffer is the async FIFO. Its gray read pointer, already synchronized into clk_link, counts freed slots. It lags, so credits return late, never early.

Q How did you avoid protocol deadlock?

Separate REQ and RSP credits, plus credits spent once at enqueue, so a replay never needs new buffer space.

Q What limits the clock frequency?

The 112-bit parallel CRC on the send and receive paths. Register the CRC check, or compute it per 32-bit beat.

Reproduce everything

```
make image    # build the tool container
make lint     # Verilator -Wall + Verible
make unit     # block-level cocotb tests
make regress  # all tests, 2 seeds, 15-min cap
make formal   # SymbiYosys proofs
make syn      # Yosys + SKY130 estimates
```

Honest limitations

Throughput

Die B runs one transaction at a time, and each AXI beat uses a whole flit.

Scope

Only INCR bursts verified; both dies assumed to reset together; FAILED needs software.

Faults

Injected errors stay within CRC-16's guaranteed detection envelope.

SV UVM

uvm_sv/ is syntax-checked only; it is meant to run on EDA Playground.

THREE TAKEAWAYS

Design for loss

Cumulative ACKs and credits make every control field self-healing.

Cross clocks one way

Eight gray-pointer FIFOs, and nothing else, cross between domains.

Back every claim

Each number traces to a log: regression, coverage, formal, synthesis.